PUSTAKA MODUL AHLI KERTAMARGA BALI | KODE MATKUL: IF101 | ID: PDF-IF-01

Buku Ajar & Praktikum: Algoritma, Struktur Data, Kompleksitas Big-O & Prinsip OOP SOLID (Standar BNSP J.620100)

Kategori Dokumen: Modul Utama Rekayasa Perangkat Lunak • **Standar Sertifikasi:** BNSP Junior Software Developer (J.620100.017.02)
Penyusun & Referensi: Tim Kurikulum Ilmu Komputer KertaMarga
Rujukan Dokumentasi Eksternal: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Data_structures
Lokasi Penyimpanan Terpusat: `public/pustaka-modul/02-informatika/MODUL-IF-01-Algoritma-Struktur-Data-Kompleksitas-BigO-dan-OOP-SOLID.pdf`

Ringkasan Eksekutif Modul: Pembahasan mendalam analisis kompleksitas waktu dan ruang Big-O, struktur data Hash Map, Binary Search Tree, Graph, serta implementasi lima prinsip desain objek SOLID dalam TypeScript.

BAB 1: Analisis Kompleksitas Algoritma (Time & Space Complexity Big-O)

Efisiensi perangkat lunak pada skala jutaan pengguna ditentukan oleh pertumbuhan jumlah operasi terhadap ukuran masukan (n), yang dinyatakan dalam notasi asimtotik Big-O. Algoritma pencarian linear (Linear Search) pada array tak terurut memiliki kompleksitas O(n), sedangkan Binary Search pada array terurut memangkas ruang pencarian separuh di setiap iterasi sehingga memiliki kompleksitas logaritmik O(log n).

Penggunaan nested loop (perulangan bersarang dua tingkat) menghasilkan kompleksitas kuadratik O(n^2) yang akan menyebabkan bottleneck CPU ketika n melampaui 10.000 baris data. Solusi rekayasa perangkat lunak modern adalah mengganti pencarian bersarang dengan struktur data Hash Map / Set yang memiliki waktu akses rata-rata konstan O(1).

```
PERBANDINGAN KOMPLEKSITAS ALGORITMA:
O(1)       -> Akses indeks Array / Lookup Hash Map (Paling Cepat)
O(log n)   -> Binary Search pada 1.000.000 data hanya butuh ~20 langkah
O(n)       -> Single Pass Loop / Filter / Reduce
O(n log n) -> Merge Sort / Quick Sort / TimSort
O(n^2)     -> Bubble Sort / Nested Loop Join (Hindari pada dataset besar)

// Optimasi O(n^2) menjadi O(n) menggunakan Map di TypeScript:
function matchStudentScores(students: {id: string, name: string}[], scores: {studentId: string, val:
number}[]) {
  const scoreMap = new Map<string, number>(scores.map(s => [s.studentId, s.val]));
  return students.map(st => ({ ...st, score: scoreMap.get(st.id) ?? 0 }));
}
```

BAB 2: Pemrograman Berorientasi Objek (OOP) & Lima Prinsip SOLID

Empat pilar OOP meliputi Encapsulation (menyembunyikan state internal dengan modifier private/protected), Inheritance (pewarisan sifat), Polymorphism (kemampuan antarmuka yang sama dieksekusi berbeda oleh tiap subclass), dan Abstraction. Untuk mencegah kode menjadi kaku dan rapuh (spaghetti code), arsitek perangkat lunak menerapkan prinsip SOLID: 1. Single Responsibility Principle (SRP): Satu kelas hanya memiliki satu alasan untuk berubah. 2. Open/Closed Principle (OCP): Terbuka untuk ekstensi, tertutup untuk modifikasi kode lama. 3. Liskov Substitution Principle (LSP): Subtipe harus dapat menggantikan tipe dasarnya tanpa merusak perilaku program. 4. Interface Segregation Principle (ISP): Antarmuka spesifik lebih baik daripada satu antarmuka raksasa. 5. Dependency Inversion Principle (DIP): Modul tingkat tinggi bergantung pada abstraksi (interface), bukan implementasi konkret.

```
// Penerapan Dependency Inversion Principle (DIP) & Open/Closed Principle:
export interface PaymentGateway {
  charge(amountIdr: number, orderId: string): Promise<{ success: boolean; txRef: string }>;
}

export class QrisBaliGateway implements PaymentGateway {
```

```
  async charge(amountIdr: number, orderId: string) {
    return { success: true, txRef: `QRIS-${orderId}-${Date.now()}` };
  }
}

export class CheckoutService {
  constructor(private readonly gateway: PaymentGateway) {}
  async processOrder(orderId: string, total: number) {
    if (total <= 0) throw new Error('Nominal transaksi tidak valid');
    return await this.gateway.charge(total, orderId);
  }
}
```

BAGIAN LANJUTAN: URAIAN LENGKAP MODUL KURIKULUM & PRAKTIKUM MATA KULIAH (IF101)

Modul 1 [SKKNI-J620100.004 - Semester 1 / Fondasi Pemrograman]: Modul 1: Pola Pikir Algoritmik, Variabel, Tipe Data, dan Struktur Kontrol (Dasar - 45 Menit)

Ringkasan Kompetensi: Memahami cara kerja komputer mengeksekusi instruksi langkah demi langkah untuk menyelesaikan masalah nyata.

Konsep Inti yang Wajib Dikuasai:

1. Deklarasi Variabel & Tipe Data Primitif (String, Number, Boolean, Null/Undefined)
2. Percabangan Kondisional (If-Else & Switch) untuk memodelkan aturan keputusan bisnis
3. Iterasi / Perulangan (For, While, ForEach) untuk memproses koleksi data berurutan
4. Penyusunan Pseudocode & Flowchart sebelum menulis sintaks bahasa pemrograman

Uraian Materi Mendalam: Algoritma bukanlah bahasa pemrograman itu sendiri, melainkan urutan langkah logis yang terukur untuk mengubah input menjadi output. Dalam mata kuliah IF101, kesalahan paling umum bagi siswa yang nilainya masih kecil adalah langsung menulis kode tanpa menggambar alur logika terlebih dahulu. Gunakan prinsip Input -> Proses -> Output. Contohnya, saat menghitung diskon koperasi siswa: (1) Baca total belanja, (2) Jika total >= 100.000 maka kurangi 10%, selain itu 0%, (3) Tampilkan harga akhir.

Studi Kasus Industri & Implementasi Bali: Studi Kasus Bali: Otomatisasi perhitungan distribusi air irigasi Subak berdasarkan luas petak sawah anggota (krama subak) menggunakan fungsi perulangan dan proporsi.

```
Instruksi Praktikum Mandiri: Buat alur logika 5 langkah untuk menentukan apakah seorang siswa berhak menerima beasiswa prestasi berdasarkan rata-rata rapor >= 82 dan kehadiran >= 90%.
```

Modul 2 [SKKNI-J620100.017 - Semester 2 / Struktur Data & OOP]: Modul 2: Struktur Data (Array, Stack, Queue, Hash Map) & Kompleksitas Big-O (Menengah - 55 Menit)

Ringkasan Kompetensi: Memilih struktur penyimpanan di memori yang tepat agar program tetap cepat saat memproses puluhan ribu data.

Konsep Inti yang Wajib Dikuasai:

1. Kompleksitas Waktu Big-O: O(1) Konstan, O(log n) Binary Search, O(n) Linear Search, dan O(n^2) Nested Loop
2. Stack (LIFO - Last In First Out, contoh: fitur Undo/Redo) vs Queue (FIFO - First In First Out, contoh: antrean cetak dokumen)
3. Hash Map / Dictionary (Key-Value Pair): Pencarian data instan O(1) berdasarkan kunci unik seperti NISN atau Kode Produk
4. Empat Pilar OOP: Encapsulation, Abstraction, Inheritance, dan Polymorphism

Uraian Materi Mendalam: Memilih struktur data yang salah akan membuat aplikasi melambat drastis saat jumlah pengguna bertambah. Jika Anda mencari satu siswa dari 50.000 data menggunakan pencarian berurutan (Linear

Search pada Array tidak terurut), komputer harus memeriksa hingga 50.000 kali (O(n)). Dengan menyimpan data di dalam Hash Map menggunakan NISN sebagai key, pencarian selesai dalam 1 langkah langsung (O(1)).

Studi Kasus Industri & Implementasi Bali: Studi Kasus Bali: Implementasi struktur data Queue (FIFO) pada sistem pemesanan tiket antrean kendaraan pelabuhan Gilimanuk dan Padangbai.

```
Instruksi Praktikum Mandiri: Jelaskan struktur data apa (Stack atau Queue) yang paling tepat digunakan untuk memproses antrean pesanan makanan di dapur restoran, beserta alasannya!
```

BAGIAN EVALUASI: BANK SOAL & PEMBAHASAN SERTIFIKASI (BNSP Junior Software Developer (J.620100.017.02))

Soal Sertifikasi #1 [Unit Kompetensi: J.620100.009.01 | Skema: Oracle Database SQL & Data Structure]

Pertanyaan Kasus: Seorang pengembang ingin menyimpan daftar 10.000 produk UMKM Bali di memori agar dapat dicari berdasarkan kode unik SKU dengan waktu akses paling cepat O(1). Struktur data mana yang paling efisien digunakan?

A. [KUNCI JAWABAN] Hash Map / Dictionary dengan kunci (key) berupa kode unik SKU produk

B. Variabel tunggal bertipe String yang digabung dengan tanda koma

C. Perulangan bersarang (Nested Loop) tiga tingkat setiap kali mencari

D. Menyimpan seluruh data di dalam komentar HTML halaman utama

```
Pembahasan Teknis Asesor: Benar! Dalam mata kuliah Algoritma & Struktur Data serta unit kompetensi J.620100.009.01, Hash Map / Dictionary memungkinkan pencarian berdasarkan kunci unik dengan kompleksitas waktu rata-rata O(1) atau konstan.
```

Soal Sertifikasi #2 [Unit Kompetensi: J.620100.017.02 | Skema: Oracle Database SQL & Data Structure]

Pertanyaan Kasus: Jika sebuah algoritma pengurutan memiliki dua perulangan `for` bersarang (nested loop) yang masing-masing berjalan sebanyak `n` elemen data, berapakah kompleksitas waktu Big-O pada skenario terburuknya?

A. O(1) - Waktu konstan

B. O(log n) - Waktu logaritmik

C. [KUNCI JAWABAN] O(n^2) - Waktu kuadratik

D. O(0) - Tanpa eksekusi CPU

```
Pembahasan Teknis Asesor: Tepat! Dua perulangan bersarang di mana perulangan luar berjalan `n` kali dan perulangan dalam berjalan `n` kali untuk setiap iterasi menghasilkan `n * n = n^2` operasi atau O(n^2).
```